الاستدعاء الذاتي

المدة
٢٠ دقيقة

المواد
هاتف وعليه هذا الدرس بعد تنزيله، ثلاث قطع نقدية مختلفة الحجم لأبراج هانوي (اختياري)

أهداف التعلّم

- تكتب دوال تستدعي نفسها بحالة أساسية وحالة تكرارية.
- تتتبّع مكدّس الاستدعاءات وتعرف حدوده.
- تسرّع العمل المتكرّر بتذكّر النتائج.

النشاط: مكدّس الاستدعاءات البشري

١. التمهيد (٣ دقائق): يقف خمسة أشخاص في صف. يسأل كلٌّ منهم مَن خلفه: «كم شخصًا خلفك؟» حتى يقول الأخير: «لا أحد». ثم تعود الأجوبة على طول الصف.

٢. التمثيل (٧ دقائق): مثّلوا fact(4): كل شخص استدعاء واحد، يكتب قيمة n الخاصة به، وينتظر حتى يعيد إليه الشخص التالي جوابًا. والشخص الأخير (n = 1) هو الحالة الأساسية.

٣. العدّ (٧ دقائق): ارسموا على الورق شجرة استدعاءات fib(5)، وحوّطوا كل استدعاء يكرّر عملًا أُنجز في مكان آخر. كم مرة تظهر fib(2)؟

٤. الإصلاح (٣ دقائق): أضيفوا ورقة مشتركة للأجوبة المحسوبة. وأعيدوا رسم الشجرة مع شطب الاستدعاءات التي يمكن البحث عنها في الورقة. كم استدعاءً بقي؟

شجرة الاستدعاءات الورقية تُظهر العمل المتكرّر قبل تشغيل أيّ كود.

للنقاش

- متى تكون الحلقة أفضل من الاستدعاء الذاتي؟
- لماذا يكرّر fib كل هذا العمل؟
- أيّ المسائل من حولك مصنوعة من نسخ أصغر منها؟

تحقّق من نفسك

- اكتب الحالة الأساسية والحالة التكرارية لدالة تجمع قائمة.

جرّبها هذا الأسبوع

أوقف خمسة أصدقاء في صف. يسأل كلٌّ منهم الذي خلفه: «كم شخصًا خلفك؟» حتى يقول الأخير: «لا أحد». ثم تعود الأجوبة على طول الصف: هذا هو الاستدعاء الذاتي، ومعه الحالة الأساسية.

ورقة عمل

الاستدعاء الذاتي

البرمجة، المستوى ٤، الدرس ٢

الاسم _____ الفصل _____ التاريخ _____

١ تحتفظ Fib(30) التي تتذكّر النتائج بقاموس للأجوبة التي حسبتها. كم مرة تحسب Fib(2)؟

○ نحو نصف مليون مرة ○ مرة واحدة ○ مرتين

٢ اكتب digit_sum(n) بالاستدعاء الذاتي: تعيد مجموع أرقام العدد الصحيح n ≥ 0. الحالة الأساسية: الرقم الواحد مجموعه هو نفسه. والحالة التكرارية: آخر رقم زائد digit_sum لما تبقّى.

```
def digit_sum(n):
    # base case: one digit (n < 10)
    # recursive case: n % 10 + digit_sum(n // 10)
    return 0
```

اكتب برنامجك:

٣ رتّب دالة المضروب بالاستدعاء الذاتي، ثم اطبع fact(5). وفيها سطر لا مكان له.

رقّم الأسطر بالترتيب (١، ٢، ٣...)، وارسم أمام السطر سهمًا ← عن كل إزاحة يحتاج إليها. اشطب أي سطر لا ينتمي إلى البرنامج.

☐ `return 1`

☐ `return n * fact(n)`

☐ `print(fact(5))`

☐ `def fact(n):`

☐ `return n * fact(n - 1)`

☐ `if n == 1:`

٤ يجب أن تطبع countdown(3) القيم 3 و2 و1 ثم تتوقّف، لكنها لا تصل أبدًا إلى حالتها الأساسية. أيّ سطر فيه الخطأ؟

ضع دائرة حول السطر الذي فيه الخطأ، ثم اكتبه صحيحًا:

```
1  def countdown(n):
2      if n == 10:
3          return
4      print(n)
5      countdown(n - 1)
6
7  countdown(3)
```

٥ تحتاج أبراج هانوي إلى $2^n - 1$ نقلة لـ n قرص. كم نقلة لخمسة أقراص؟

○ 31 ○ 25 ○ 32

✎ أين ترى مسألة مصنوعة من نسخ أصغر منها (مجلّدات داخل مجلّدات، أو شجرة عائلة، أو بطولة)؟ صِف حالتها الأساسية وحالتها التكرارية.

تحقّق من إجاباتك

الاستدعاء الذاتي

البرمجة، المستوى ٤، الدرس ٢

١ **مرة واحدة**

مع تذكّر النتائج تُحسب كل قيمة مرة واحدة، ثم يُبحث عنها في القاموس.

٢

```
def digit_sum(n):
    if n < 10:
        return n
    return n % 10 + digit_sum(n // 10)
```

كل استدعاء يقتطع آخر رقم ويستدعي نفسه على ما تبقّى، أي عدد أصغر، حتى يبقى رقم واحد: الحالة الأساسية.

٣ **من الأعلى إلى الأسفل: ٣، ✗، ٥، ١، ٤، ٢**

```
def fact(n):
    if n == 1:
        return 1
    return n * fact(n - 1)
print(fact(5))
```

تأتي الحالة الأساسية أولًا، وتستدعي الحالة التكرارية fact على n − 1، مسألة أصغر.

٤ **السطر 2:** `if n == 0:`

تصير n: 3 ثم 2 ثم 1 ثم 0 ثم −1... ولا تساوي 10 أبدًا، فالحالة الأساسية لا يمكن الوصول إليها، وتتراكم الاستدعاءات حتى ينفد المكدّس.

٥ **31**

$2^5 - 1 = 32 - 1 = 31$. كل قرص إضافي يضاعف النقلات ويزيد واحدًا: V لثلاثة أقراص، و١٥ لأربعة، و٣١ لخمسة.